

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники**

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
дисциплины «Искусственный интеллект и машинное обучение»**

Выполнила:

Федорова Дарья Юрьевна 2 курс,
группа ИВТ-б-о-24-1, 09.03.01
«Информатика и вычислительная
техника», направленность
(профиль) «Программное
обеспечение средств
вычислительной техники и
автоматизированных систем»,
очная форма обучения

(подпись)

Руководитель практики:
Воронкин Р. А.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

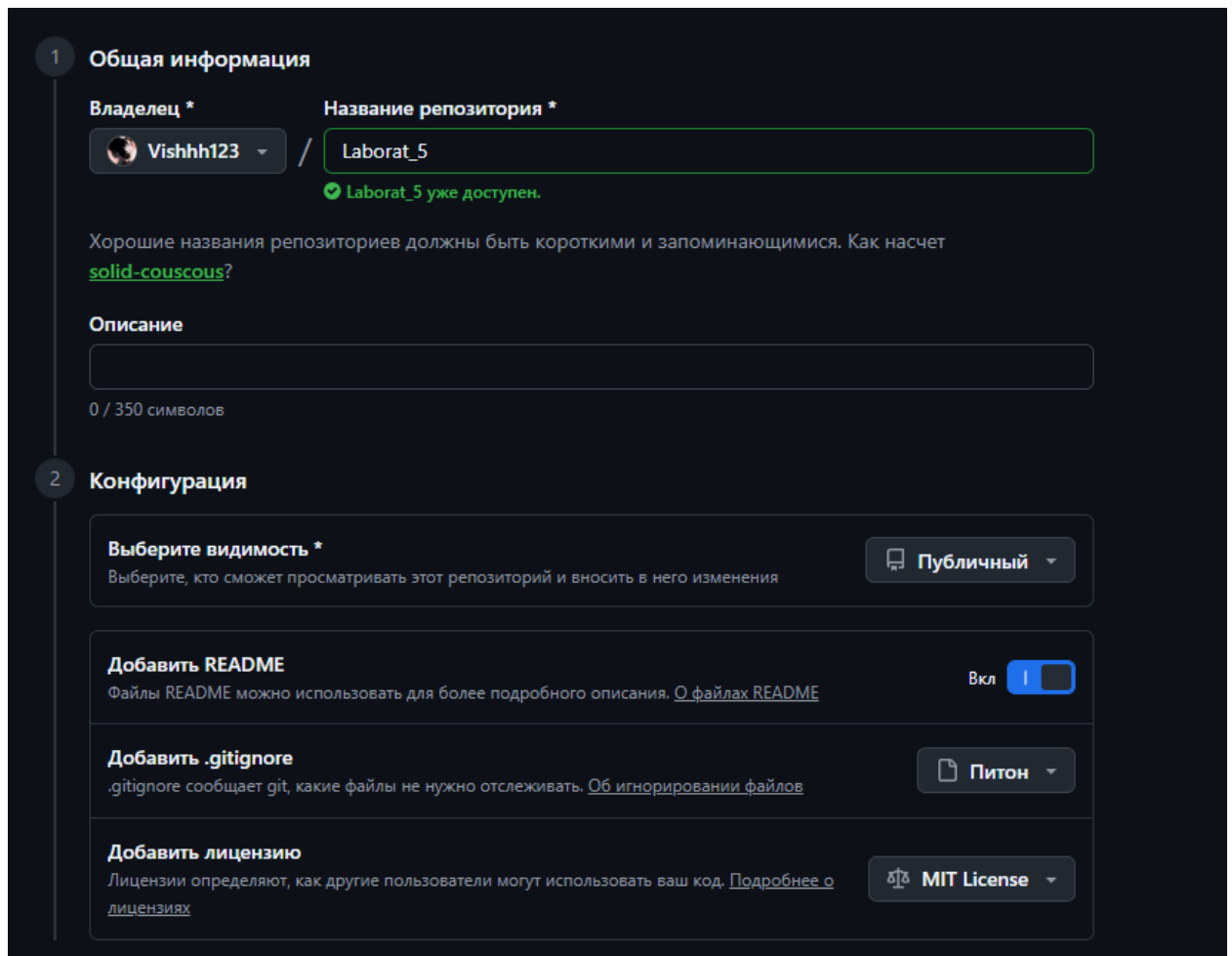
Ставрополь, 2025 г.

Вариант 25

Тема: Введение в pandas: изучение структуры DataFrame и базовых операций
Цель: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных DataFrame.

Ссылка на репозиторий: https://github.com/Vishhh123/Laborat_5

Ход работы:



1 **Общая информация**

Владелец * / Название репозитория *

Vishhh123 / Laborat_5

✓ Laborat_5 уже доступен.

Хорошие названия репозитория должны быть короткими и запоминающимися. Как насчет [solid-couscous?](#)

Описание

0 / 350 символов

2 **Конфигурация**

Выберите видимость *

Выберите, кто сможет просматривать этот репозиторий и вносить в него изменения

Публичный

Добавить README

Файлы README можно использовать для более подробного описания. [О файлах README](#)

Вкл

Добавить .gitignore

.gitignore сообщает git, какие файлы не нужно отслеживать. [Об игнорировании файлов](#)

Питон

Добавить лицензию

Лицензии определяют, как другие пользователи могут использовать ваш код. [Подробнее о лицензиях](#)

MIT License

Рис. 1 - создание общедоступного репозитория с лицензией MIT, языком программирования Python.

```
PS C:\Users\user> git clone https://github.com/Vishhh123/Laborat_5.git
Cloning into 'Laborat_5'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
PS C:\Users\user> cd Laborat_5
```

Рис. 2 - клонирование репозитория на устройство, переход к репозиторию.

Ход работы:

```
In [6]: import pandas as pd

s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s)
```

a	10
b	20
c	30
d	40

dtype: int64

```
In [7]: data = {
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
}

df = pd.DataFrame(data, index=["Анна", "Иван", "Ольга"])
print(df)
```

	Возраст	Город
Анна	25	Москва
Иван	30	СПб
Ольга	22	Казань

```
In [8]: print(df["Возраст"])
```

Анна	25
Иван	30
Ольга	22

Name: Возраст, dtype: int64

```
In [10]: s = pd.Series([10, 20, 30], index=["a", "b", "c"])
df = s.to_frame(name="Значение")
print(df)
```

	Значение
a	10
b	20
c	30

```
In [11]: s = df["Значение"]
print(s)
```

a	10
b	20
c	30

Name: Значение, dtype: int64

Рис. 3 – Примеры создания Series и DataFrame, извлечение столбцов и преобразование Series в DataFrame.

In [12]:

```
import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

In [13]:

```
ages = pd.Series([25, 30, 22], index=["a", "b", "c"])
cities = pd.Series(["Москва", "СПб", "Казань"], index=["a", "b", "c"])

df = pd.DataFrame({"Возраст": ages, "Город": cities})
print(df)
```

	Возраст	Город
a	25	Москва
b	30	СПб
c	22	Казань

In [14]:

```
data = [
    ["Анна", 25, "Москва"],
    ["Иван", 30, "СПб"],
    ["Ольга", 22, "Казань"]
]

df = pd.DataFrame(data, columns=["Имя", "Возраст", "Город"])
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рис. 4 – Создание DataFrame из словаря, из Series и из списка списков.

```
In [16]: numeric_data = np.array([
        [25, 65.5],
        [30, 78.2],
        [22, 54.3]
    ])

df = pd.DataFrame(numeric_data, columns=["Возраст", "Вес"])
print(df)
```

	Возраст	Вес
0	25.0	65.5
1	30.0	78.2
2	22.0	54.3

```
In [17]: df = pd.DataFrame(columns=["Имя", "Возраст", "Город"])
print(df)
```

Empty DataFrame
Columns: [Имя, Возраст, Город]
Index: []

```
In [18]: df.loc[0] = ["Анна", 25, "Москва"]
df.loc[1] = ["Иван", 30, "СПб"]
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб

```
In [21]: import pandas as pd

df = pd.read_csv("C:/Users/user/Laborat_5/Дополнительно/data.csv")
print(df.head())
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань
3	Петр	40	Новосибирск
4	Мария	35	Екатеринбург

```
In [23]: df = pd.read_csv("C:/Users/user/Laborat_5/Дополнительно/data.csv", sep=";", index_col=0, na_values=["?", "N/A"])
print(df)
```

Empty DataFrame
Columns: []
Index: [Анна,25,Москва, Иван,30,СПб, Ольга,22,Казань, Петр,40,Новосибирск, Мария,35,Екатеринбург]

```
In [24]: df.to_csv("C:/Users/user/Laborat_5/Дополнительно/output.csv", index=False)
```

Рис. 5 – Создание DataFrame из массива NumPy, пустого DataFrame с последующим добавлением строк, чтение из CSV и запись в CSV.

```
In [27]: df = pd.read_excel("C:/Users/user/Laborat_5/Дополнительно/data.xlsx", sheet_name="Лист1")
print(df.head())
```

	Имя	Возраст	Город
0	Анна	64	Москва
1	Иван	17	СПб
2	Ольга	12	Казань
3	Петр	33	Новосибирск
4	Мария	34	Екатеринбург

```
In [28]: df.to_excel("C:/Users/user/Laborat_5/Дополнительно/output.xlsx", sheet_name="Результаты", index=False)
```

```
In [40]: import sqlite3

conn = sqlite3.connect("Дополнительно/database.db")
query = "SELECT * FROM users"
df = pd.read_sql(query, conn)
print(df.head())
```

	id	name	age	city
0	1	Анна	25	Москва
1	2	Иван	30	СПб
2	3	Ольга	22	Казань

```
In [46]: df = pd.read_json("C:/Users/user/Laborat_5/Дополнительно/dataa.json")
print(df.head())
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
In [48]: df.to_json("Дополнительно/output.json", orient="records", indent=4)
```

```
In [50]: pip install pyarrow
```

Requirement already satisfied: pyarrow in C:\Users\user\anaconda3\Lib\site-packages (21.0.0)
Note: you may need to restart the kernel to use updated packages.

Рис. 6 – Чтение данных из Excel, SQLite и JSON, запись в Excel и JSON, установка библиотеки pyarrow для работы с Parquet.

```
In [51]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "Екатеринбург"]
}

df = pd.DataFrame(data)
df.to_parquet("Дополнительно/data.parquet", engine="pyarrow", index=False)
print("Файл data.parquet создан!")
```

Файл data.parquet создан!

```
In [52]: df.to_parquet("Дополнительно/output.parquet", engine="pyarrow", index=False)
```

```
In [53]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Сергей"],
    "Возраст": [25, 30, 22, 40, 35, 28],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "Екатеринбург", "Сочи"]
}

df = pd.DataFrame(data)

print(df.head(3))
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
In [54]: print(df.tail(2))
```

	Имя	Возраст	Город
4	Мария	35	Екатеринбург
5	Сергей	28	Сочи

```
In [55]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Имя      6 non-null      str
1   Возраст  6 non-null      int64
2   Город    6 non-null      str
dtypes: int64(1), str(2)
memory usage: 416.0 bytes
```

Рис. 7 – Работа с форматом Parquet (запись данных), а также использование методов `.head(3)`, `.tail(2)` и `.info()` для первичного анализа DataFrame.

```
In [56]: print(df.describe())
```

	Возраст
count	6.00000
mean	30.00000
std	6.60303
min	22.00000
25%	25.75000
50%	29.00000
75%	33.75000
max	40.00000

```
In [59]: print(df.describe(include="str"))
```

	Имя	Город
count	6	6
unique	6	6
top	Анна	Москва
freq	1	1

```
In [60]: import pandas as pd
```

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр"],
    "Возраст": [25, 30, 22, 40],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск"]
}

df = pd.DataFrame(data, index=["a", "b", "c", "d"])
print(df)
```

	Имя	Возраст	Город
a	Анна	25	Москва
b	Иван	30	СПб
c	Ольга	22	Казань
d	Петр	40	Новосибирск

```
In [61]: print(df.loc["b"])
```

Имя	Иван
Возраст	30
Город	СПб

Name: b, dtype: object

```
In [63]: print(df.loc[["a", "c"]])
```

	Имя	Возраст	Город
a	Анна	25	Москва
c	Ольга	22	Казань

Рис. 8 – Статистический анализ данных через `.describe()`, а также примеры доступа к строкам `DataFrame` с помощью `.loc[]` по меткам индекса.


```
In [65]: print(df.loc[:, ["Имя", "Город"]])
```

	Имя	Город
a	Анна	Москва
b	Иван	СПб
c	Ольга	Казань
d	Петр	Новосибирск

```
In [66]: print(df.loc[df["Возраст"] > 25])
```

	Имя	Возраст	Город
b	Иван	30	СПб
d	Петр	40	Новосибирск

```
In [67]: print(df.iloc[1])
```

	Имя	Иван
Возраст		30
Город		СПб

Name: b, dtype: object

```
In [68]: print(df.iloc[2, 2])
```

Казань

```
In [69]: print(df.iloc[[0, 2]])
```

	Имя	Возраст	Город
a	Анна	25	Москва
c	Ольга	22	Казань

```
In [70]: print(df.iloc[1:3])
```

	Имя	Возраст	Город
b	Иван	30	СПб
c	Ольга	22	Казань

```
In [71]: print(df.iloc[:, [0, 2]])
```

	Имя	Город
a	Анна	Москва
b	Иван	СПб
c	Ольга	Казань
d	Петр	Новосибирск

Рис. 9 – Примеры доступа к данным с помощью `.loc[]` (по меткам) и `.iloc[]` (по номерам): выборка строк и столбцов, фильтрация по условию и срезы.

```
In [72]: print(df.at["c", "Город"])
```

Казань

```
In [73]: print(df.iat[2, 2])
```

Казань

Рис. 10 – Быстрый доступ к одному элементу с помощью `.at[]` (по меткам) и `.iat[]` (по номерам).

In [1]: `import pandas as pd`

```
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22]
}

df = pd.DataFrame(data)
df["Город"] = ["Москва", "СПб", "Казань"]

print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

In [2]: `df["Страна"] = "Россия"`
`print(df)`

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия

In [3]: `df["Возраст_в_месяцах"] = df["Возраст"] * 12`
`print(df)`

	Имя	Возраст	Город	Страна	Возраст_в_месяцах
0	Анна	25	Москва	Россия	300
1	Иван	30	СПб	Россия	360
2	Ольга	22	Казань	Россия	264

In [4]: `df = df.assign(Зарплата=[50000, 60000, 45000])`
`print(df)`

	Имя	Возраст	Город	Страна	Возраст_в_месяцах	Зарплата
0	Анна	25	Москва	Россия	300	50000
1	Иван	30	СПб	Россия	360	60000
2	Ольга	22	Казань	Россия	264	45000

In [5]: `df["Категория возраста"] = df["Возраст"].apply(lambda x: "Молодой" if x < 30 else "Взрослый")`
`print(df)`

	Имя	Возраст	Город	Страна	Возраст_в_месяцах	Зарплата	\
0	Анна	25	Москва	Россия	300	50000	
1	Иван	30	СПб	Россия	360	60000	
2	Ольга	22	Казань	Россия	264	45000	

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой

Рис. 11 – Добавление новых столбцов в DataFrame: через прямое присваивание, на основе вычислений, с помощью `.assign()` и через `.apply()` с лямбда-функцией.

```
In [7]: df.loc[3] = ["Петр", 40, "Новосибирск", "Россия", 480, 85000, "Взрослый"]
print(df)
```

	Имя	Возраст	Город	Страна	Возраст_в_месяцах	Зарплата	\
0	Анна	25	Москва	Россия	300	50000	
1	Иван	30	СПб	Россия	360	60000	
2	Ольга	22	Казань	Россия	264	45000	
3	Петр	40	Новосибирск	Россия	480	85000	

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой
3	Взрослый

```
In [9]: new_row = pd.DataFrame([{"Имя": "Сергей", "Возраст": 35, "Город": "Сочи", "Страна": "Россия"}])
df = pd.concat([df, new_row], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна	Возраст_в_месяцах	Зарплата	\
0	Анна	25	Москва	Россия	300.0	50000.0	
1	Иван	30	СПб	Россия	360.0	60000.0	
2	Ольга	22	Казань	Россия	264.0	45000.0	
3	Петр	40	Новосибирск	Россия	480.0	85000.0	
4	Сергей	35	Сочи	Россия	NaN	NaN	

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой
3	Взрослый
4	NaN

```
In [10]: new_data = pd.DataFrame([{"Имя": "Алексей", "Возраст": 29, "Город": "Томск", "Страна": "Россия"}])
df = pd.concat([df, new_data], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна	Возраст_в_месяцах	Зарплата	\
0	Анна	25	Москва	Россия	300.0	50000.0	
1	Иван	30	СПб	Россия	360.0	60000.0	
2	Ольга	22	Казань	Россия	264.0	45000.0	
3	Петр	40	Новосибирск	Россия	480.0	85000.0	
4	Сергей	35	Сочи	Россия	NaN	NaN	
5	Алексей	29	Томск	Россия	NaN	NaN	

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой
3	Взрослый
4	NaN
5	NaN

Рис. 12 — Добавление строк в DataFrame: через .loc[] (с полным списком значений) и через pd.concat() (с добавлением новых записей в виде DataFrame).

```
In [11]: new_rows = pd.DataFrame([
    {"Имя": "Дмитрий", "Возраст": 31, "Город": "Самара", "Страна": "Россия"},
    {"Имя": "Елена", "Возраст": 27, "Город": "Воронеж", "Страна": "Россия"}
])

df = pd.concat([df, new_rows], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна	Возраст_в_месяцах	Зарплата \
0	Анна	25	Москва	Россия	300.0	50000.0
1	Иван	30	СПб	Россия	360.0	60000.0
2	Ольга	22	Казань	Россия	264.0	45000.0
3	Петр	40	Новосибирск	Россия	480.0	85000.0
4	Сергей	35	Сочи	Россия	NaN	NaN
5	Алексей	29	Томск	Россия	NaN	NaN
6	Дмитрий	31	Самара	Россия	NaN	NaN
7	Елена	27	Воронеж	Россия	NaN	NaN

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой
3	Взрослый
4	NaN
5	NaN
6	NaN
7	NaN

```
In [14]: df.loc["новый"] = {"Имя": "Артем", "Возраст": 33, "Город": "Калуга"}
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань
новый	Артем	33	Калуга

```
In [13]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"],
    "Зарплата": [50000, 60000, 45000]
}

df = pd.DataFrame(data)

df = df.drop(columns=["Зарплата"])
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рис. 13 – Добавление нескольких строк через `pd.concat()`, добавление строки по метке через `.loc[]`, а также удаление столбца с помощью `.drop(columns=[...])`.

```
In [15]: df = df.drop(columns=['Возраст', 'Город'])
print(df)
```

	Имя
0	Анна
1	Иван
2	Ольга
новый	Артем

```
In [16]: del df["Имя"]
print(df)
```

Empty DataFrame
Columns: []
Index: [0, 1, 2, новый]

```
In [18]: df = pd.DataFrame(data)
df = df.drop(index=1)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
2	Ольга	22	Казань	45000

```
In [19]: df = df.drop(index=[0, 2])
print(df)
```

Empty DataFrame
Columns: [Имя, Возраст, Город, Зарплата]
Index: []

```
In [20]: df = pd.DataFrame(data)
df = df[df["Возраст"] > 25]
print(df)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

```
In [21]: df = df.reset_index(drop=True)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Иван	30	СПб	60000

```
In [22]: df.loc[3] = ["Мария", None, "Самара", 55000]
df = df.dropna()
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Иван	30	СПб	60000

Рис. 14 – Удаление столбцов и строк: `.drop(columns)`, `del`, `.drop(index)`, фильтрация по условию, сброс индекса и удаление строк с пропусками через `.dropna()`.

```
In [23]: df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Анна", "Ольга"],
    "Возраст": [25, 30, 25, 22]
})

df = df.drop_duplicates()
print(df)
```

	Имя	Возраст
0	Анна	25
1	Иван	30
3	Ольга	22

```
In [24]: df = df.drop_duplicates(subset=['Имя'])
```

```
In [25]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
In [26]: df_filtered = df.query("Возраст > 30")
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
In [27]: df_filtered = df.query("Город == 'СПб' and Зарплата > 60000")
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
4	Мария	35	СПб	65000

Рис. 15 – Удаление дубликатов через `.drop_duplicates()`, а также фильтрация данных с помощью `.query()` по одному и нескольким условиям.

```
In [28]: min_age = 25
df_filtered = df.query("Возраст > @min_age")
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
In [29]: df_filtered = df[df["Город"].isin(["Москва", "СПб"])]
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000

```
In [30]: df_filtered = df[~df["Город"].isin(["Москва", "СПб"])]
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000

```
In [31]: df_filtered = df[df["Возраст"].between(25, 35)]
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000

```
In [32]: df_filtered = df[df["Возраст"].between(25, 35, inclusive="neither")]
print(df_filtered)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

Рис. 16 – Фильтрация данных: с использованием переменной в `.query()` через `@`, а также методы `.isin()` (включая инверсию с `~`) и `.between()` с разными параметрами включения границ.

```
In [33]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", None],
    "Возраст": [25, 30, 22, None, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"]
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город
0	Анна	25.0	Москва
1	Иван	30.0	СПб
2	Ольга	22.0	Казань
3	Петр	NaN	Новосибирск
4	NaN	35.0	СПб

```
In [34]: print(df.count())
```

```
Имя      4
Возраст  4
Город    5
dtype: int64
```

```
In [35]: print(df["Город"].value_counts())
```

```
Город
СПб      2
Москва   1
Казань   1
Новосибирск 1
Name: count, dtype: int64
```

```
In [36]: print(df["Город"].value_counts(sort=False))

print(df["Имя"].value_counts(dropna=False))
```

```
Город
Москва   1
СПб      2
Казань   1
Новосибирск 1
Name: count, dtype: int64

Имя
Анна     1
Иван     1
Ольга    1
Петр     1
NaN      1
Name: count, dtype: int64
```

Рис. 17 – Работа с пропусками и подсчёт значений: `.count()` для непустых значений, `.value_counts()` с сортировкой и с учётом NaN через `dropna=False`.


```
In [37]: print(df.nunique())
```

```
Имя      4
Возраст   4
Город     4
dtype: int64
```

```
In [38]: print(df["Имя"].nunique(dropna=False))
```

```
5
```

```
In [39]: print(df.isna().sum())
```

```
Имя      1
Возраст   1
Город     0
dtype: int64
```

```
In [40]: import pandas as pd
```

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Зарплата
0	Анна	25	50000
1	Иван	30	60000
2	Ольга	22	45000
3	Петр	40	70000
4	Мария	35	65000

```
In [41]: df_sorted = df.sort_values(by="Возраст")
print(df_sorted)
```

	Имя	Возраст	Зарплата
2	Ольга	22	45000
0	Анна	25	50000
1	Иван	30	60000
4	Мария	35	65000
3	Петр	40	70000

Рис. 18 – Подсчёт уникальных значений (.nunique()) с учётом NaN, определение пропусков (.isna().sum()), а также сортировка данных по столбцу "Возраст" с помощью .sort_values().

```
In [42]: df_sorted = df.sort_values(by="Возраст", ascending=False)
print(df_sorted)
```

	Имя	Возраст	Зарплата
3	Петр	40	70000
4	Мария	35	65000
1	Иван	30	60000
0	Анна	25	50000
2	Ольга	22	45000

```
In [43]: df_sorted = df.sort_values(by=["Возраст", "Зарплата"], ascending=[True, False])
print(df_sorted)
```

	Имя	Возраст	Зарплата
2	Ольга	22	45000
0	Анна	25	50000
1	Иван	30	60000
4	Мария	35	65000
3	Петр	40	70000

```
In [44]: df.loc[5] = ["Елена", None, 55000]
df_sorted = df.sort_values(by="Возраст", na_position="first")
print(df_sorted)
```

	Имя	Возраст	Зарплата
5	Елена	None	55000
2	Ольга	22	45000
0	Анна	25	50000
1	Иван	30	60000
4	Мария	35	65000
3	Петр	40	70000

```
In [45]: df_sorted = df.sort_index()
print(df_sorted)
```

	Имя	Возраст	Зарплата
0	Анна	25	50000
1	Иван	30	60000
2	Ольга	22	45000
3	Петр	40	70000
4	Мария	35	65000
5	Елена	None	55000

Рис. 19 – Сортировка данных: по убыванию (`ascending=False`), по нескольким столбцам, с перемещением NaN в начало (`na_position="first"`), а также сортировка по индексу (`.sort_index()`).

```
In [46]: df_sorted = df.sort_index(ascending=False)
print(df_sorted)
```

	Имя	Возраст	Зарплата
5	Елена	None	55000
4	Мария	35	65000
3	Петр	40	70000
2	Ольга	22	45000
1	Иван	30	60000
0	Анна	25	50000

```
In [47]: import pandas as pd

df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
})

print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
In [48]: from IPython.display import display
display(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
In [49]: df.head(5)

df.tail(3)
```

```
Out[49]:
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рис. 20 – Сортировка индекса по убыванию (.sort_index(ascending=False)), создание DataFrame и отображение с помощью display() для красивого вывода в Jupyter.

```
In [50]: pd.set_option("display.max_rows", 100)

pd.set_option("display.max_columns", 50)

pd.set_option("display.max_colwidth", None)
```

```
In [51]: pd.set_option("display.float_format", "{:.2f}".format)

df.style.set_sticky()
```

```
Out[51]:
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рис. 21 – Настройка параметров отображения DataFrame: изменение максимального числа строк и столбцов, отключение обрезки текста, форматирование чисел с плавающей точкой и фиксация заголовков через .set_sticky().

```
In [ ]: from tabulate import tabulate
print(tabulate(df, headers="keys", tablefmt="github"))

In [54]: df.style.set_properties(**{"background-color": "lightgray", "color": "black"})

html_table = df.to_html()
with open("table.html", "w", encoding="utf-8") as f:
    f.write(html_table)
```

Рис. 22 – Вывод DataFrame в виде таблицы с помощью библиотеки tabulate и экспорт в HTML-формат с применением стилей через .style.set_properties().

```
In [2]: import pandas as pd
import numpy as np

data = {
    "ID": [1,2,3,4,5],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей"],
    "Возраст": [25,30,40,35,28],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000],
    "Стаж": [2,5,15,7,3]
}
df1 = pd.DataFrame(data)
```

```
In [3]: # Устанавливаю ID как индекс
df = df1.set_index("ID")
print(df)
```

	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
ID						
1	Иван	25	Инженер	IT	60000	2
2	Ольга	30	Аналитик	Маркетинг	75000	5
3	Алексей	40	Менеджер	Продажи	90000	15
4	Мария	35	Программист	IT	80000	7
5	Сергей	28	Специалист	HR	50000	3

```
In [4]: # 3.1 .loc[] - сотрудник с ID=5
print("\nСотрудник с ID=5:")
print(df.loc[5])
```

```
Сотрудник с ID=5:
Имя          Сергей
Возраст      28
Должность    Специалист
Отдел        HR
Зарплата     50000
Стаж         3
Name: 5, dtype: object
```

```
In [5]: # 3.2 .iloc[] - возраст третьего сотрудника
print("\nВозраст третьего сотрудника:")
print(df.iloc[2, 2])
```

```
Возраст третьего сотрудника:
Менеджер
```

```
In [6]: # 3.3 .at[] - отдел Марии
print("\nОтдел Марии:")
print(df.at[4, "Отдел"])
```

```
Отдел Марии:
IT
```

Рис. 23 – Задание 3: доступ к данным с использованием .loc[] (поиск сотрудника по ID), .iloc[] (возраст третьего сотрудника) и .at[] (отдел Марии).

```
In [7]: # 3.4 .iat[] - зарплата в 4-й строке, 5-м столбце
print("\nЗарплата:")
print(df.iat[3, 5])
```

Зарплата:
7

Рис. 24 — Задание 3: доступ к элементу с помощью `.iat[]` — получение зарплат в 4-й строке и 5-м столбце (значение 7).

Задания:

1. Создание DataFrame разными способами

1. Создайте DataFrame из словаря списков с данными о пяти сотрудниках (возьмите из таблицы 1).
2. Создайте DataFrame из списка словарей с теми же данными.
3. Создайте DataFrame из массива NumPy, заполненного случайными числами от 20 до 60 (для возраста сотрудников).
4. Проверьте типы данных в каждом столбце с помощью `.info()`.

```
In [7]: #1. Создание DataFrame
# 1.1 Из словаря
import pandas as pd
import numpy as np

data = {
    "ID": [1,2,3,4,5],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей"],
    "Возраст": [25,30,40,35,28],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000],
    "Стаж": [2, 5, 15, 7, 3]
}
df1 = pd.DataFrame(data)
print(df1)
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

```
In [8]: # 1.2 Из списка словарей
data2 = [
    {"ID":1, "Имя": "Иван", "Возраст": 25, "Должность": "Инженер", "Отдел": "IT", "Зарплата": 60000, "Стаж": 2},
    {"ID":2, "Имя": "Ольга", "Возраст": 30, "Должность": "Аналитик", "Отдел": "Маркетинг", "Зарплата": 75000, "Стаж": 5}
]
df2 = pd.DataFrame(data2)
print(df2)
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5

```
In [9]: # 1.3 Из NumPy (случайный возраст)
np.random.seed(1)
age = np.random.randint(20,61,(5,1))
df3 = pd.DataFrame(age, columns=["Возраст"])
print(df3)
```

	Возраст
0	57
1	32
2	28
3	29
4	31

Рис. 25 – код к заданию 1

```
In [10]: # 1.4 Типы данных
df1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0    ID           5 non-null      int64
1   Имя           5 non-null      str
2  Возраст       5 non-null      int64
3  Должность     5 non-null      str
4   Отдел        5 non-null      str
5  Зарплата      5 non-null      int64
6   Стаж         5 non-null      int64
dtypes: int64(4), str(3)
memory usage: 592.0 bytes
```

Рис. 26 – код к заданию 1.

2. Чтение данных из файлов (CSV , Excel , JSON)

Используйте предоставленные таблицы и сохраните их в файлы перед чтением.

1. Сохраните **Таблицу 1** в **CSV** и затем загрузите ее обратно в **DataFrame** .
2. Запишите **Таблицу 2** в **Excel** (**data.xlsx** , лист **"Клиенты"**) и прочитайте ее в **DataFrame** .
3. Экспортируйте **Таблицу 1** в формат **JSON** и затем прочитайте ее обратно в **DataFrame** .

Выведите первые 5 строк загруженных **DataFrame** .

```
In [16]: import os
os.makedirs("Дополнительно", exist_ok=True)

In [18]: import pandas as pd
import numpy as np
import os

# Создаю папку Дополнительно
os.makedirs("Дополнительно", exist_ok=True)

# Создаю df1
data = {
    "ID": [1,2,3,4,5],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей"],
    "Возраст": [25,30,40,35,28],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000],
    "Стаж": [2,5,15,7,3]
}
df1 = pd.DataFrame(data)

# 2.1 CSV
df1.to_csv("Дополнительно/table1.csv", index=False, encoding="utf-8")
df_csv = pd.read_csv("Дополнительно/table1.csv")
print("CSV:")
print(df_csv.head())

# 2.2 Excel
df1.to_excel("Дополнительно/table1.xlsx", index=False)
df_excel = pd.read_excel("Дополнительно/table1.xlsx")
print("\nExcel:")
print(df_excel.head())

# 2.3 JSON
df1.to_json("Дополнительно/table1.json", orient="records", force_ascii=False)
df_json = pd.read_json("Дополнительно/table1.json")
print("\nJSON:")
print(df_json.head())
```

CSV:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Excel:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

JSON:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рис. 27 – код к заданию 2.

3. Доступ к данным (`.loc` , `.iloc` , `.at` , `.iat`)

Используя Таблицу 1:

1. Получите информацию о сотруднике с ID = 5 с помощью `.loc[]` .
2. Выведите возраст **третьего сотрудника** в таблице с `.iloc[]` .
3. Выведите название **отдела** для сотрудника "Мария" с `.at[]` .
4. Выведите **зарплату** сотрудника, находящегося в **четвертой строке и пятом столбце**, используя `.iat[]` .

Объясните разницу между `.loc[]` , `.iloc[]` , `.at[]` , `.iat[]` .

```
[13]: import pandas as pd

data = {
    "ID": [1,2,3,4,5],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей"],
    "Возраст": [25,30,40,35,28],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000],
    "Стаж": [2,5,15,7,3]
}
df = pd.DataFrame(data)
df = df.set_index("ID")
```

```
[14]: print("DataFrame:")
print(df)
print()

DataFrame:
   Имя  Возраст  Должность  Отдел  Зарплата  Стаж
ID
1  Иван     25  Инженер    IT      60000     2
2  Ольга     30  Аналитик  Маркетинг  75000     5
3  Алексей    40  Менеджер  по продажам  90000    15
4  Мария     35  Программист  IT      80000     7
5  Сергей     28  Специалист  по кадрам  50000     3
```

```
[15]: print("3.1 Сотрудник с ID=5:")
print(df.loc[5])
print()

3.1 Сотрудник с ID=5:
Имя  Сергей
Возраст  28
Должность  Специалист
Отдел  кадров
Зарплата  50000
Стаж  3
Имя: 5, тип данных: объект
```

Рис. 28 – код к заданию 3


```
[16]: print("3.2 Возраст третьего сотрудника:")  
print(df.iloc[2, 2])  
print()
```

3.2 Возраст третьего сотрудника:
Менеджер

```
[17]: print("3.3 Отдел Марии:")  
print(df.at[4, "Отдел"])  
print()
```

3.3 Отдел Марии:
IT

```
[18]: print("3.4 Зарплата в 4-й строке, 5-м столбце:")  
print(df.iat[3, 5])
```

3.4 Зарплата в 4-й строке, 5-й столбец:
7

Рис. 29 – код к заданию 3

4. Добавление новых столбцов и строк

Используя **Таблицу 1**:

1. Добавьте новый столбец "Категория зарплаты" :

- "Низкая" – если Зарплата < 60000 .
- "Средняя" – если $60000 \leq \text{Зарплата} < 100000$.
- "Высокая" – если Зарплата ≥ 100000 .

2. Добавьте нового сотрудника с данными:



```
1 df.loc[21] = ["Антон", 32, "Разработчик", "IT", 85000, 6]
```

3. Добавьте **двух новых сотрудников** одновременно, используя `pd.concat()` .

Выведите обновленный `DataFrame` .

```
[4]: # 4.1 Добавляю столбец "Категория зарплаты"
def category(salary):
    if salary < 60000:
        return "Низкая"
    elif salary < 100000:
        return "Средняя"
    else:
        return "Высокая"

df1["Категория зарплаты"] = df1["Зарплата"].apply(category)
print("4.1 Столбец 'Категория зарплаты' добавлен:")
print(df1[["Имя", "Зарплата", "Категория зарплаты"]])
print()

4.1 Добавлен столбец «Категория зарплаты»:
Имя Зарплата Категория зарплаты
0 Иван 60000 Средняя
1 Ольга 75000 Средняя
2 Алексей 90000 Средняя
3 Мария 80000 Средняя
4 Сергей 50000 Низкая
```

```
[5]: # 4.2 Добавляю одного сотрудника через .loc[]
df1.loc[21] = [21, "Антон", 32, "Разработчик", "IT", 85000, 6, "Средняя"]
print("4.2 Добавлен сотрудник Антон:")
print(df1.tail())
print()

4.2 Добавлен сотрудник Антон:
ID Имя Возраст Должность Отдел Зарплата Стаж \
1 2 Ольга 30 Аналитик Маркетинг 75000 5
2 3 Алексей 40 Менеджер по продажам 90000 15
3 4 Мария 35 Программист IT 80000 7
4 5 Сергей 28 Специалист по кадрам 50000 3
21 21 Антон 32 Разработчик IT 85000 6

Категория зарплаты
1 Средняя
2 Средняя
3 Средняя
4 Низкая
21 Средняя
```

Рис. 30 – код к заданию 4

```
[6]: # 4.3 Добавляю двух сотрудников через pd.concat()
new_employees = pd.DataFrame([
    {"ID": 22, "Имя": "Максим", "Возраст": 29, "Должность": "Аналитик", "Отдел": "Маркетинг", "Зарплата": 72000, "Стаж": 4, "Категория": "Средняя"},
    {"ID": 23, "Имя": "Наталья", "Возраст": 36, "Должность": "Менеджер", "Отдел": "Продажи", "Зарплата": 92000, "Стаж": 10, "Категория": "Средняя"}
])
df1 = pd.concat([df1, new_employees], ignore_index=True)
print("4.3 Добавлены два сотрудника (Максим и Наталья):")
print(df1.tail())

4.3 Добавлены два сотрудника (Максим и Наталья):
ID Имя Возраст Должность Отдел Зарплата Стаж \
3 4 Мария 35 Программист IT 80000 7
4 5 Сергей 28 Специалист HR 50000 3
5 21 Антон 32 Разработчик IT 85000 6
6 22 Максим 29 Аналитик Маркетинг 72000 4
7 23 Наталья 36 Менеджер Продажи 92000 10

Категория зарплаты
3 Средняя
4 Низкая
5 Средняя
6 Средняя
7 Средняя
```

Рис. 31 – код к заданию 4

5. Удаление строк и столбцов

Используя **Таблицу 1**:

1. Удалите столбец "Категория зарплаты" с `.drop()` .
2. Удалите строку с `ID = 10` .
3. Удалите **все строки**, где `Стаж работы < 3 лет` .
4. Удалите **все столбцы**, кроме `Имя` , `Должность` , `Зарплата` .

Выведите `DataFrame` после каждого шага.

```
[3]: # 5.1 Удалила столбец "Категория зарплаты"
df1 = df1.drop(columns=["Категория зарплаты"])
print("5.1 Столбцы после удаления:")
print(df1.columns)
print()

5.1 Столбцы после удаления:
Index(['ID', 'Имя', 'Возраст', 'Должность', 'Отдел', 'Зарплата', 'Стаж'], dtype='str')
```

```
[4]: # 5.2 Удалила строку с ID = 10
df1 = df1[df1["ID"] != 10]
print("5.2 После удаления ID=10:")
print(df1)
print()

5.2 После удаления ID=10:
  ID  Имя  Возраст  Должность  Отдел  Зарплата  Стаж
0  1  Иван    25  Инженер  IT  60000  2
1  2  Ольга   30  Аналитик  Маркетинг  75000  5
2  3  Алексей  40  Менеджер по продажам  90000  15
3  4  Мария   35  Программист  IT  80000  7
4  5  Сергей  28  Специалист по кадрам  50000  3
5  21  Антон   32  Разработчик  IT  85000  6
6  22  Максим  29  Аналитик  Маркетинг  72000  4
7  23  Наталья  36  Менеджер по продажам  92000  10
```

```
[5]: # 5.3 Удалила строки со стажем < 3 лет
df1 = df1[df1["Стаж"] >= 3]
print("5.3 После удаления стажа < 3:")
print(df1)
print()

5.3 После удаления стажа < 3:
  ID  Имя  Возраст  Должность  Отдел  Зарплата  Стаж
1  2  Ольга   30  Аналитик  Маркетинг  75000  5
2  3  Алексей  40  Менеджер по продажам  90000  15
3  4  Мария   35  Программист  IT  80000  7
5  21  Антон   32  Разработчик  IT  85000  6
6  22  Максим  29  Аналитик  Маркетинг  72000  4
7  23  Наталья  36  Менеджер по продажам  92000  10
```

Рис. 32 — код к заданию 5

```
[6]: # 5.4 Оставила только Имя, Должность, Зарплата
df1 = df1[["Имя", "Должность", "Зарплата"]]
print("5.4 Только Имя, Должность, Зарплата:")
print(df1)
```

5.4 Только имя, должность, зарплата:

Имя Должность Зарплата

1 Ольга Аналитик 75000

2 Алексей Менеджер 90000

3 Мария Программист 80000

4 Сергей Специалист 50000

5 Антон Разработчик 85000

6 Максим Аналитик 72000

7 Наталья Менеджер 92000

Рис. 33 – код к заданию 5

6. Фильтрация данных (query , isin , between)

Используя **Таблицу 2**:

1. Выберите всех клиентов из "Москва" или "Санкт-Петербург" , используя `.isin()` .
2. Выберите клиентов, у которых **Баланс на счете от 100000 до 250000**, используя `.between()` .
3. Отфильтруйте клиентов, у которых "Кредитная история" "Хорошая" и "Баланс на счете" > 150000 , используя `.query()` .

Выведите результаты каждого фильтра.

```
[11]: import pandas as pd
import numpy as np

data2 = {
    "ID": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Алиса", "Павел", "Светлана", "Роман", "Татьяна",
    "Возраст": [34,27,45,38,29,50,31,40,28,33,46,37,41,25,39,42,49,50,30,35],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Екатеринбург", "Воронеж", "Челябинск", "Краснодар", "Ростов-на-Дону", "
    "Баланс на счете": [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 98000, 250000, 210000, 135000, 155000, 125000, 180000
    "Кредитная история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная", "Средняя", "Хорошая", "Плохая", "Средняя", "Хорош

}

df2 = pd.DataFrame(data2)

[12]: # 6.1 Клиенты из Москвы или Санкт-Петербурга
print("Клиенты из Москвы или СПб:")
print(df2[df2["Город"].isin(["Москва", "Санкт-Петербург"])])
print()

Клиенты из Москвы или Санкт-Петербурга:
  ID Имя Возраст Город Остаток на счете Кредитная история
0 1 Иван 34 Москва 120000 Хорошая
1 2 Ольга 27 Санкт-Петербург 80000 Средняя

[13]: # 6.2 Клиенты с балансом от 100000 до 250000
print("Баланс от 100000 до 250000:")
print(df2[df2["Баланс на счете"].between(100000, 250000)])
print()

Остаток от 100000 до 250000:
  ID Имя Возраст Город Остаток на счете Кредитная история
0 1 Иван 34 Москва 120000 Хорошая
2 3 Алексей 45 Казань 150000 Плохая
3 4 Мария 38 Новосибирск 200000 Хорошая
6 7 Дмитрий 31 Челябинск 140000 Средняя
7 8 Елена 40 Краснодар 175000 Хорошая
8 9 Виктор 28 Ростов-на-Дону 110000 Плохая
10 11 Павел 46 Омск 250000 Хорошая
11 12 Светлана 37 Пермь 210000 Отличная
12 13 Роман 41 Тюмень 135000 Средняя
13 14 Татьяна 25 Саратов 155000 Хорошая
14 15 Николай 39 Самара 125000 Средняя
15 16 Валерия 42 Волгоград 180000 Плохая
18 19 Степан 30 Хабаровск 105000 Средняя
```

Рис. 34 – код к заданию 6

```
[14]: # 6.3 Хорошая история и баланс > 150000
print("Кредитная история Хорошая и баланс > 150000:")
print(df2.query("`Кредитная история` == 'Хорошая' and `Баланс на счете` > 150000"))

Кредитная история Хорошая и баланс > 150000:
  ID Имя Возраст Город Баланс на счете Кредитная история
3 4 Мария 38 Новосибирск 200000 Хорошая
7 8 Елена 40 Краснодар 175000 Хорошая
10 11 Павел 46 Омск 250000 Хорошая
13 14 Татьяна 25 Саратов 155000 Хорошая
17 18 Юлия 50 Иркутск 320000 Хорошая
```

Рис. 35 – код к задана 6

7. Подсчет значений (count, value_counts, nunique)

Используя Таблицу 2:

1. Подсчитайте количество **непустых значений** в каждом столбце (`.count()`).
2. Определите **частоту встречаемости значений** в "Город" (`.value_counts()`).
3. Найдите **количество уникальных значений** в "Город" , "Возраст" , "Баланс на счете" (`.nunique()`).

Выведите результаты.

```
[2]: import pandas as pd
import numpy as np

data2 = {
    "ID": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Алиса", "Павел", "Светлана", "Роман", "Татьяна",
    "Возраст": [34,27,45,38,29,50,31,40,28,33,46,37,41,25,39,42,49,50,30,35],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Екатеринбург", "Воронеж", "Челябинск", "Краснодар", "Ростов-на-Дону", "
    "Баланс на счете": [120000,80000,150000,200000,95000,300000,140000,175000,110000,98000,250000,210000,135000,155000,125000,18000
    "Кредитная история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная", "Средняя", "Хорошая", "Плохая", "Средняя", "Хорош
}

df2 = pd.DataFrame(data2)

[3]: # 7.1 Количество непустых значений
print("Непустые значения:")
print(df2.count())
print()

Непустые значения:
ID      20
Имя     20
Возраст 20
Город   20
Остаток на счете 20
Кредитная история 20
dtype: int64
```

Рис. 36 – код к заданию 7

```
[4]: # 7.2 Частота городов
print("Частота городов:")
print(df2["Город"].value_counts())
print()

Частота городов:
Город
Москва 1
Санкт-Петербург 1
Казань 1
Новосибирск 1
Екатеринбург 1
Воронеж 1
Челябинск 1
Краснодар 1
Ростов-на-Дону 1
Уфа 1
Омск 1
Пермь 1
Тюмень 1
Саратов 1
Самара 1
Волгоград 1
Барнаул 1
Иркутск 1
Хабаровск 1
Томск 1
Имя: count, тип данных: int64

[5]: # 7.3 Уникальные значения
print("Уникальных значений:")
print("Город:", df2["Город"].nunique())
print("Возраст:", df2["Возраст"].nunique())
print("Баланс:", df2["Баланс на счете"].nunique())

Уникальных значений:
Город: 20
Возраст: 19
Баланс: 20
```

Рис. 37 – код к заданию 7

8. Обнаружение пропусков (`isna` , `notna`)

Используя **Таблицу 3**:

1. Подсчитайте **количество** `NaN` в каждом столбце (`.isna().sum()`).
2. Подсчитайте **количество заполненных значений** в каждом столбце (`.notna().sum()`).
3. Выведите `DataFrame` , оставив только строки, где **нет пропущенных значений**.

Объясните разницу между `.isna()` и `.notna()` .

```
[2]: import pandas as pd

data3 = {
    "ID": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", None, "Дмитрий", "Елена", "Виктор", "Алиса", "Павел", None, "Роман", "Татьяна", "Никол",
    "Возраст": [34.0, 27.0, None, 38.0, 29.0, 50.0, 31.0, 40.0, None, 33.0, 46.0, 37.0, 41.0, 25.0, None, 42.0, 49.0, None, 30.0, 35.0],
    "Город": ["Москва", "Санкт-Петербург", "Казань", None, "Екатеринбург", "Воронеж", None, "Краснодар", "Ростов-на-Дону", "Уфа", "Омск", "Пер",
    "Баланс на счете": [120000.0, None, 150000.0, 200000.0, None, 300000.0, 140000.0, 175000.0, 110000.0, None, 250000.0, 210000.0, 135000.0, 15
    "Кредитная история": ["Хорошая", "Средняя", "Плохая", "Хорошая", None, "Отличная", "Средняя", "Хорошая", None, "Средняя", "Хорошая", "Отли

}

df3 = pd.DataFrame(data3)

print(df3.head())
```

```
ID Имя Возраст Город Остаток на счете Кредитная история
0 1 Иван 34.0 Москва 120000.0 Хорошая
1 2 Ольга 27.0 Санкт-Петербург NaN Средняя
2 3 Алексей NaN Казань 150000.0 Плохая
3 4 Мария 38.0 NaN 200000.0 Хорошая
4 5 Сергей 29.0 Екатеринбург NaN NaN
```

```
[3]: # 8.1 Количество NaN
print("Количество NaN:")
print(df3.isna().sum())
print()
```

```
Количество NaN:
ID 0
Имя 2
Возраст 4
Город 4
Остаток на счете 4
Кредитная история 3
dtype: int64
```

Рис. 38 – код к заданию 8

```
[4]: # 8.2 Количество заполненных
print("Количество заполненных:")
print(df3.notna().sum())
print()
```

```
Количество заполненных:
ID 20
Имя 18
Возраст 16
Город 16
Остаток на счете 16
Кредитная история 17
dtype: int64
```

```
[5]: # 8.3 Строки без пропусков
print("Строки без пропусков:")
print(df3.dropna())
```

```
Строки без пропусков:
ID Имя Возраст Город Остаток на счете Кредитная история
0 1 Иван 34.0 Москва 120000.0 Хорошая
7 8 Елена 40.0 Краснодар 175000.0 Хорошая
10 11 Павел 46.0 Омск 250000.0 Хорошая
16 17 Григорий 49.0 Барнаул 275000.0 Отличная
18 19 Степан 30.0 Хабаровск 105000.0 Средняя
19 20 Василиса 35.0 Томск 90000.0 Плохая
```

Рис. 39 – код к заданию 8


```
] import pandas as pd
from datetime import datetime

df = pd.DataFrame(columns=["Пункт", "Номер", "Время"])

while True:
    print("\n1 - Добавить 2 - Найти 3 - Удалить 4 - Сохранить 5 - Загрузить 6 - Выход")
    cmd = input("Выбор: ")

    if cmd == "1":
        p = input("Пункт: ")
        n = input("Номер: ")
        t = input("Время (ЧЧ:ММ): ")
        df.loc[len(df)] = [p, n, t]
        df = df.sort_values("Пункт")
        print("Добавлено!")

    elif cmd == "2":
        t = input("Время (ЧЧ:ММ): ")
        res = df[df["Время"] > t]
        if res.empty:
            print("Нет поездов после", t)
        else:
            print(res)

    elif cmd == "3":
        col = input("Какую колонку удалить? (Пункт/Номер/Время): ")
        if col in df.columns:
            df = df.drop(columns=[col])
            print("Удалено!")
        else:
            print("Нет такой колонки")

    elif cmd == "4":
        df.to_parquet("trains.parquet", index=False)
        print("Сохранено!")

    elif cmd == "5":
        try:
            df = pd.read_parquet("trains.parquet")
            print("Загружено!")
        except:
            print("Файл не найден")

    elif cmd == "6":
        break

1 - Добавить 2 - Найти 3 - Удалить 4 - Сохранить 5 - Загрузить 6 - Выход
Выбор: 1
Пункт: Москва
Номер: 123
Время (ЧЧ:ММ): 15:30
Добавлено!

1 - Добавить 2 - Найти 3 - Удалить 4 - Сохранить 5 - Загрузить 6 - Выход
```

Рис. 40 — код к индивидуальному заданию

Ответы на контрольные вопросы:

1. Как создать `pandas.DataFrame` из словаря списков?

Передать словарь в `pd.DataFrame()`, где ключи — названия столбцов, значения — списки.

2. В чем отличие создания `DataFrame` из списка словарей и словаря списков?

Список словарей — каждая запись отдельно, удобно если есть пропуски. Словарь списков — все списки должны быть одинаковой длины.

3. Как создать `pandas.DataFrame` из массива `NumPy`?

`pd.DataFrame(np_array, columns=[...])`

4. Как загрузить DataFrame из CSV-файла, указав разделитель ;?

`pd.read_csv("file.csv", sep=";")`

5. Как загрузить данные из Excel в pandas.DataFrame и выбрать конкретный лист?

`pd.read_excel("file.xlsx", sheet_name="Лист1")`

6. Чем отличается чтение данных из JSON и Parquet в pandas?

JSON — текстовый, человекочитаемый. Parquet — бинарный, сжатый, быстрее для больших данных.

7. Как проверить типы данных в DataFrame после загрузки?

`df.info()` или `df.dtypes`

8. Как определить размер DataFrame (количество строк и столбцов)?

`df.shape`

9. В чем разница между .loc[] и .iloc[]?

.loc[] — по названиям (меткам). .iloc[] — по номерам (индексам).

10. Как получить данные третьей строки и второго столбца с .iloc[]?

`df.iloc[2, 1]`

11. Как получить строку с индексом "Мария" из DataFrame?

`df.loc["Мария"]`

12. Чем .at[] отличается от .loc[]?

.at[] — быстрее, только для одного элемента. .loc[] — для нескольких строк/столбцов.

13. В каких случаях .iat[] работает быстрее, чем .iloc[]?

При доступе к одному элементу. .iat[] оптимизирован для одиночного значения.

14. Как выбрать все строки, где "Город" равен "Москва" или "СПб"?

`df[df["Город"].isin(["Москва", "СПб"])]`

15. Как отфильтровать строки, где "Возраст" от 25 до 35?

`df[df["Возраст"].between(25, 35)]`

16. В чем разница между .query() и .loc[] для фильтрации?

.query() — строковое выражение (SQL-стиль). .loc[] — булева маска.

17. Как использовать переменные Python внутри .query()?

Через @: `df.query("Возраст > @min_age")`

18. Как узнать количество пропущенных значений в каждом столбце?

`df.isna().sum()`

19. В чем разница между `.isna()` и `.notna()`?

`.isna()` — показывает True для NaN. `.notna()` — показывает True для заполненных.

20. Как вывести только строки без пропусков?

`df.dropna()`

21. Как добавить столбец "Категория" со значением "Неизвестно"?

`df["Категория"] = "Неизвестно"`

22. Как добавить строку через `.loc[]`?

`df.loc[индекс] = [значения]`

23. Как удалить столбец "Возраст"?

`df.drop(columns=["Возраст"])`

24. Как удалить все строки с NaN?

`df.dropna()`

25. Как удалить столбцы с NaN?

`df.dropna(axis=1)`

26. Как посчитать количество непустых значений в каждом столбце?

`df.count()`

27. Чем `.value_counts()` отличается от `.unique()`?

`.value_counts()` — частота каждого значения. `.unique()` — только список уникальных.

28. Как определить частоту значений в столбце "Город"?

`df["Город"].value_counts()`

29. Почему `display(df)` лучше, чем `print(df)`, в Jupyter Notebook?

`display(df)` — красивая HTML-таблица. `print(df)` — текст, хуже читается.

30. Как изменить максимальное количество строк в Jupyter?

`pd.set_option("display.max_rows", 100)`

Вывод: В ходе работы были изучены структура и основные операции с DataFrame в pandas: создание из разных источников, чтение/запись файлов (CSV, Excel, JSON, Parquet), фильтрация, добавление и удаление данных.